rmw_unix_socket_cpp

A purpose-built ROS 2 middleware for single-machine deployments

DRIX Ocean — Autonomous Vessel Platform

Abderahmane Benali

What this talk is about

A custom **RMW** (ROS 2 Middleware) implementation that replaces DDS for our 150+ node production system on the DRIX vessel.

~3000 lines of C++. Zero external middleware. Linux kernel primitives only.

We'll cover:

1. The problem: why DDS was hurting us
2. What an RMW actually is
3. The four design decisions that defined the project
4. Trade-offs we accepted (and the ones that bit us)
5. Where we are today

The context: DRIX Ocean

- **Autonomous surface vessel** for hydrographic survey and ocean mapping
- **Single onboard computer** runs the entire stack — perception, control, comms, logging
- **150+ ROS 2 nodes** in production
- **Docker-based deployment** — every subsystem in its own container
- **No network ROS** — everything is on one machine

> Single-host, high-node-count, containerized. The default DDS deployment was not built for this.

Why DDS was hurting us

Symptom	Root cause
30+ second startup before topics matched	DDS multicast discovery storm with 150 nodes
~5 GB RAM just for middleware	Per-participant DDS state, multicast sockets, internal buffers
Mysterious "topic not visible" issues across containers	DDS multicast doesn't cross Docker network namespaces cleanly
Hundreds of UDP sockets per process	One per remote endpoint
CPU spikes during graph changes	DDS discovery protocol re-floods on every join/leave

DDS is excellent for distributed, multi-machine fleets. **It is overkill for one box.**

What is an RMW?

ROS 2 is built in **layers**:

```
+-----------------------------------------+
|  Application code (rclcpp / rclpy)      |
+-----------------------------------------+
|  rcl   (C client library)               |
+-----------------------------------------+
|  rmw   (abstract middleware ABI)        |  <-- swappable
+-----------------------------------------+
|  rmw_<impl>   (FastDDS, Cyclone, ...) |
+-----------------------------------------+
```

The **rmw layer** is a fixed C ABI — ~80 functions. Anyone can implement it. ROS 2 picks the implementation at runtime via `RMW_IMPLEMENTATION=...`.

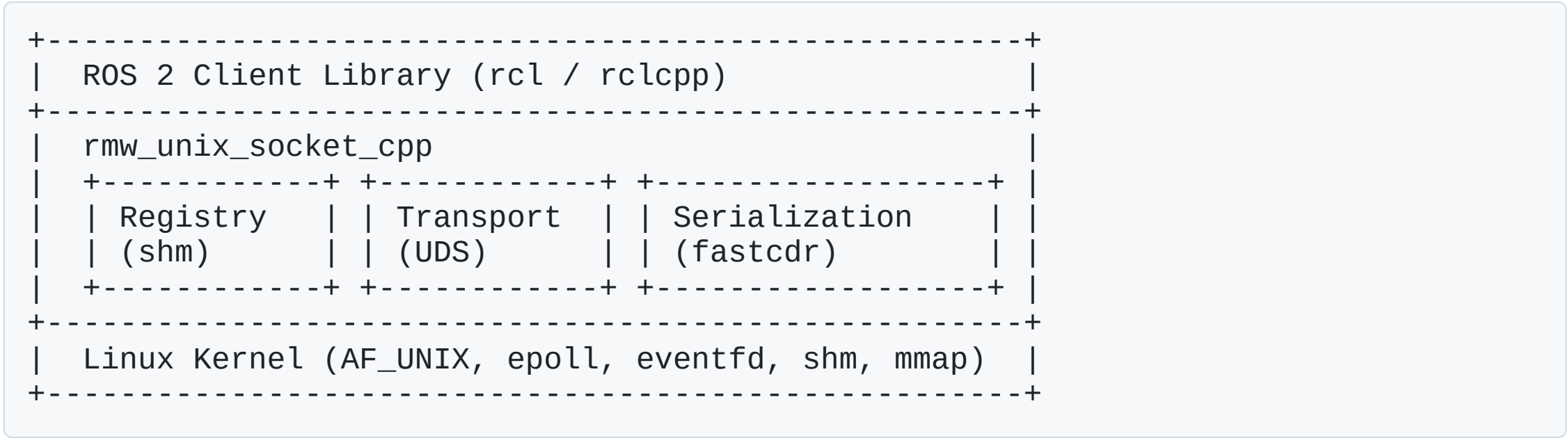
The four design decisions

Building an RMW comes down to answering four questions:

#	Question	DDS answer	Our answer
1	**Discovery**: how do peers find each other?	Multicast SPDP	Shared memory registry
2	**Transport**: how do bytes move?	RTPS over UDP/SHM	AF_UNIX `SOCK_DGRAM`
3	**Serialization**: message → bytes?	CDR	CDR (via `fastcdr`)
4	**Wait**: block on N event sources?	Internal threads + condvars	`epoll` + `eventfd`, no threads

The rest is mechanical glue.

Architecture overview



Three internal modules. No threads. No external dependencies beyond `rmw`, `pthread`, `rt`, and `fastcdr`.

Decision 1 — Discovery: shared memory registry

The graph lives in `/dev/shm/ros2_uds_<domain_id>` — a single mmap'd file.

Layout:

```
+-----------------------------------------+
| Header: version, generation, robust mutex |
+-----------------------------------------+
| RegistryEntry[0]    32768 fixed slots     |
| RegistryEntry[1]                          |
| ...                                       |
+-----------------------------------------+
```

Each `node`, `publisher`, `subscription`, `service`, `client` writes itself into one slot.

Total: ~36 MB. Visible to every process. Zero IPC for lookup.

Why shared memory (and not a daemon)?

Approach	Pros	Cons
Daemon (rosmaster-style)	Centralized, easy to reason about	Process to manage, startup ordering, single point of failure
Multicast UDP discovery (DDS)	Decentralized, scalable across hosts	Storm at scale, doesn't cross Docker networks cleanly
Shared memory ✓	No process to manage, instant lookup, simple	Single-host only, fixed-size limits

For our single-host target, shared memory is the simplest mechanism that works.

Why a fixed-size registry?

```
static constexpr uint32_t REGISTRY_MAX_ENTRIES = 32768;
```

- 32768 slots × ~1156 bytes = ~36 MB
- Avoids `mremap`, pointer invalidation, dynamic resize complexity
- Comfortably handles 200 nodes × ~10 endpoints × parameter services ≈ 4000 entries
- 8× headroom for our worst case

Lookup is a **linear scan** — at this size, a few microseconds even fully populated.

Decision 2 — Transport: SOCK_DGRAM Unix sockets

All data (topics, requests, responses) → AF_UNIX SOCK_DGRAM.

```
publisher                              subscriber
   |                                       |
   | sendmsg(/tmp/ros2_uds/0/sub_1002_1.sock)
   |--------- iovec[hdr + payload] -->|
                                           | recvmsg() in rmw_wait
                                           | drain into std::deque
                                           | rmw_take() pops + deserializes
```

One **send fd shared by all publishers** in a process; one **bound fd per subscription/service/client**.

SOCK_DGRAM vs SOCK_STREAM

Property	SOCK_DGRAM (ours)	SOCK_STREAM
Connections	None — connectionless	accept/connect per peer
Framing	Native (1 datagram = 1 msg)	DIY length-prefix + read loop
Multi-sub fanout	N independent `sendmsg`	N persistent connections to manage
Backpressure	Drop on `EAGAIN`	Block on full buffer
Big messages	Capped by `wmem_max` (4 MB)	Unlimited
Code complexity	Low	High

Trade-off: simplicity for the cost of accepting drop-on-overflow semantics (more on this later).

Why not iceoryx-style shared memory for data?

Zero-copy is tempting. Iceoryx delivers message bytes through SHM with no kernel boundary cross.

But it costs:

- Memory pools and slot management
- Reference counting across processes
- Flow control and backpressure logic
- Subscriber liveness tracking (orphan reclaim)

Unix sockets give us 90% of localhost performance with 10% of the complexity.

The kernel already handles buffering, flow control, lifetime. Re-implementing that for a marginal win didn't pay.

Decision 3 — Serialization: fastcdr (after a painful detour)

First attempt: custom introspection-based serializer walking `MessageMembers` at runtime.

It worked. For simple messages.

Production failure modes at 100+ nodes:

1. **C vs C++ ABI mismatch** — same `MessageMember` struct, different layouts
2. `std::vector<bool>` — bitfield, not contiguous, returns `nullptr` from `get_function`
3. **Nested message resolution** — wrong introspection variant returned by typesupport dispatch
4. **Every fix introduced regressions** elsewhere

We tried hard. It does not generalize.

Why fastcdr won

```
callbacks->cdr_serialize(msg, cdr);     // compiled per-message-type
callbacks->cdr_deserialize(cdr, msg);
```

- **Compiled per-message-type code —** no runtime field walking
- **Battle-tested**: used by `rmw_fastrtps_cpp`, `rmw_zenoh_cpp`, `rmw_connextdds`
- **130 lines** of serialization code (vs ~400 lines for introspection)
- **Standard ROS 2 dependency —** installed with any distro

Lesson: if the default RMWs solve a problem the same way, there's usually a reason.

Decision 4 — Wait: epoll + eventfd, no threads

Single function does all the work:

```
rmw_ret_t rmw_wait(subs, guards, services, clients, events, ws, timeout)
{
  drain_all_sockets();           // recvmsg into per-entity queues
  check_graph_generation();      // bump guard condition if changed
  if (anything_ready) return;
  epoll_ctl_add(all_fds);
  epoll_wait(timeout);
  drain_all_sockets();           // again, after wakeup
  null_out_not_ready_entries();
}
```

No background threads. No lock contention. The executor calls this in a tight loop anyway.

Why epoll, eventfd, no threads?

Choice	Reason
`epoll` over `poll`/`select`	O(1) vs O(n) on watched fds; matters at 200+ nodes
`eventfd` for guard conditions	Single 8-byte kernel counter, integrates natively with epoll
No background threads	No lock contention on message queues; no thread lifecycle bugs
Drain inside `rmw_wait`	Same approach as `rmw_zenoh_cpp`; sufficient because executor calls `wait` in a loop

The simplest design that meets the requirements.

Robustness: what happens when processes die

Production reality: nodes get SIGKILL'd, OOM'd, container-restarted. They never call destroy.

Three-level cleanup, no daemon:

1. `rmw_init` — every startup scrubs stale registry slots and orphan `/tmp/ros2_uds/` files
2. **Robust mutex** `EOWNERDEAD` — if a process died holding the registry lock, the next acquirer recovers it via `pthread_mutex_consistent`
3. `registry_add` **retry** — when slots fill, reclaim dead-PID entries before failing

Key trick: `stat("/proc/<pid>")` — if the PID isn't in our namespace, the entry is stale.

Graph change notification

Every node has a **graph guard condition** (an eventfd).

```
// In registry header
uint64_t generation;  // incremented on every add/remove

// In rmw_wait
if (current_generation != ctx->last_seen_generation) {
    rmw_trigger_guard_condition(ctx->graph_guard_condition);
}
```

Polling, not pushing. **One atomic load per `rmw_wait` call.** Detects graph changes within milliseconds because `rmw_wait` is called constantly.

No daemon. No signals. No cross-process pipes.

QoS: enforced vs accepted

Policy	Status	Why
`KEEP_LAST` + `depth`	✓ Enforced	Subscription queue capped, oldest dropped
`TRANSIENT_LOCAL`	✓ Enforced	Publisher caches last `depth` messages, replays to late-joining subs
`RELIABLE` / `BEST_EFFORT`	⚠️ Accepted, not differentiated	UDS is reliable on localhost (mostly — see "risks")
`deadline` / `lifespan` / `liveliness`	✗ Accepted, ignored	Would need timer threads — not built

Covers ~95% of ROS 2 usage. The last 5% is honestly documented as not supported.

Trade-offs we accepted

Trade-off	Cost	Why we said yes
Localhost only	No network ROS	Single-machine is our design point
Linux only	No macOS/Windows	Production target is Ubuntu
4 MB datagram cap	Big messages need sysctl tuning	Documented; tunable
Linear registry scans	O(N) lookups	Microseconds at our scale
No background threads	All I/O at executor cadence	Avoids contention bugs
No DDS interop	Cannot mix with FastDDS / Cyclone	Whole stack must use this RMW

Risks: where data loss can actually happen

The honest answer: AF_UNIX SOCK_DGRAM is reliable in transit, but our send path uses `MSG_DONTWAIT`:

```
sendmsg(send_fd, &msg, MSG_DONTWAIT | MSG_NOSIGNAL);
```

If the subscriber's recv buffer is full → `EAGAIN` → **message dropped silently**.

Three real risks:

1. **Slow subscriber** — recv buffer fills, publisher drops. No backpressure, no log.
2. **Big message** — exceeds 4 MB datagram cap, `EMSGSIZE`, dropped.
3. **Burst publisher** — > 4 MB in flight, same outcome.

`SOCK_STREAM` would block instead. We trade reliability for fanout simplicity.

Mitigations available today

- `net.core.rmem_max` / `wmem_max` **sysctl** — bigger kernel buffers = more headroom
- **QoS** `depth` — only helps the app-layer queue, not the kernel buffer
- **Slow-subscriber detection** — possible via `SIOCINQ` ioctl in `rmw_wait` (not yet implemented)
- **Drop logging** — currently silent; should at least warn (open work)

> For RELIABLE QoS to truly mean reliable, the EAGAIN path needs explicit handling. Documented honestly.

Resource usage

Resource	Usage
Shared memory	**~36 MB** total (one file, one domain)
Sockets per subscriber	1 fd + 1 file
Sockets per service/client	1 fd + 1 file each
Send socket per process	1 fd (shared by all publishers)
`epoll` fd per wait set	1
`eventfd` per guard condition	1
Background threads	**0**

For 200 nodes × 10 pub + 10 sub each: ~4000 registry entries, ~2000 socket files, ~2000 fds.

Compare to DDS: ~2 GB RAM, hundreds of UDP sockets per process.

Docker deployment

Two non-obvious requirements:

```
docker run \
  --ipc=host \                          # for /dev/shm registry
  --pid=host \                          # for stale-PID cleanup
  -v /tmp/ros2_uds:/tmp/ros2_uds \      # for socket files
  ...
```

Why `--pid=host`? Cleanup uses `stat("/proc/<pid>")`. Without it, container A would see container B's PIDs as "dead" and unlink their socket files — corruption.

Why `--ipc=host`? `/dev/shm` is per-namespace by default. The registry must be visible to all containers.

File structure (~3000 lines total)

File	Lines	Purpose
`registry.{hpp,cpp}`	~250	Shared memory discovery
`transport.{hpp,cpp}`	~250	UDS send/receive
`serialization.{hpp,cpp}`	~130	fastcdr wrapper
`rmw_wait.cpp`	~400	The core event loop
`rmw_publisher.cpp`	~280	Publish + TRANSIENT_LOCAL cache
`rmw_subscription.cpp`	~250	Subscribe + take
`rmw_service.cpp` + `rmw_client.cpp`	~500	Request/response
`rmw_node.cpp`, `rmw_init.cpp`, `rmw_graph.cpp`, …	~700	Glue

Comparable RMWs: `rmw_fastrtps` ≈ 15k lines, `rmw_cyclonedds` ≈ 10k.

What we tested

- **Unit tests —** registry, transport, serialization, init, pub/sub, services, wait, graph, QoS
- **Real production system —** 100+ nodes, complex messages (`BridgeCommFleetStatus`, `InsStatus`, `TFMessage`)
- **Docker multi-container —** `--ipc=host --pid=host` + bind mount
- **Crash recovery —** SIGKILL'd processes, robust mutex `EOWNERDEAD` paths
- **Stale cleanup —** orphan socket files, dead-PID slots

What's next

Short-term:

- Surface `EAGAIN` send failures (logging or counter)
- Slow-subscriber detection via `SIOCINQ`
- More aggressive integration tests under load

Medium-term:

- Optional `SOCK_SEQPACKET` for RELIABLE topics that genuinely need backpressure
- Per-topic statistics (drops, queue depth) accessible via the registry

Not on the roadmap:

- Network transport (use Cyclone or Zenoh for that)
- Windows / macOS support

Lessons learned

1. **Match the middleware to the deployment topology.** DDS is excellent off-host; overkill on-host.
2. **The simplest thing that works is usually the right thing.** `SOCK_DGRAM` + shared memory beats clever zero-copy schemes for our scale.
3. **Don't fight the standard ABIs.** Custom CDR via introspection failed; `fastcdr` was 1/3 the code and 100% reliable.
4. **Honest documentation > marketing.** RELIABLE-vs-BEST_EFFORT is "accepted but not differentiated", not "fully supported".
5. **Robustness lives in the cleanup paths**, not the happy path.

Questions?

`rmw_unix_socket_cpp`

Repository: `drixo_ws_ros2/rmw/rmw_unix_socket_cpp`

Design doc: `DESIGN.md`

~3000 lines · 0 background threads · 0 external middleware